LAPORAN TUGAS BESAR 2
IF4020 KRIPTOGRAFI

Cipher Fayadh

Perancangan dan Implementasi Block Cipher Modern



Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Pernyataan

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Dita Maheswari (13523125)



Naufarrel Zhafif Abhista (13523149)



I Made Wiweka Putera (13523156)



Hasri Fayadh Muqaffa (13523160)

Daftar Isi

Pernyataan **i**

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Review Block Cipher Sejenis . . . 1
1.3 Gagasan dan Pendekatan Utama . . . 3

2 Dasar Teori **5**
2.1 Konsep *Block Cipher* . . . 5
2.2 Jaringan Feistel / SPN . . . 6
2.3 *S-Box* dan Permutasi . . . 6
2.4 Mode Operasi (ECB, CBC, CFB, OFB, CTR) . . . 7
2.5 Modulasi Sigma-Delta ($\Sigma\Delta$) . . . 8
2.6 *Padding & Integrity Verification* . . . 8
2.7 *Related Works* . . . 8

3 Perancangan Block Cipher **9**
3.1 Parameter Umum . . . 9
3.2 Alur Enkripsi . . . 10
3.3 Alur Dekripsi . . . 10
3.4 Pembentukan *Key Schedule* . . . 11
3.5 *S-Box* . . . 11
3.6 Permutasi / Rotasi . . . 12
3.7 Fungsi Putaran . . . 12

4 Implementasi dan Simulasi **13**
4.1 Deskripsi Program . . . 14
4.1.1 Antarmuka Perintah dan Format Kontainer . . . 14
4.1.2 Konvensi *Exit Code* dan Keamanan Runtime . . . 15
4.2 Hasil Uji Enkripsi dan Dekripsi . . . 15
4.2.1 Berkas Teks . . . 15
4.2.2 Berkas Biner . . . 15
4.3 Penanganan Padding . . . 16
4.4 Verifikasi Integritas Data . . . 16
4.4.1 Pemisahan Kunci dan Perbandingan Waktu-Konstan . . . 17
4.4.2 Prinsip Fail-Stop . . . 17

5 Analisis Keamanan **17**
5.1 Avalanche Effect . . . 17
5.2 Entropi . . . 19
5.3 Histogram . . . 20
5.4 Pengujian Tambahan . . . 21

6 Kesimpulan dan Saran Pengembangan **23**
6.1 Kesimpulan 23
6.2 Saran Pengembangan (Future Works) 24

A Lampiran **27**
A.1 Repositori Source Code 27
A.2 Video Demo Program 27
A.3 Dokumentasi API 27
A.4 Pembagian Tugas 27

1 Pendahuluan

1.1 Latar Belakang

Block cipher merupakan komponen dasar kriptografi kunci simetri modern. Selain dipakai langsung untuk menjaga kerahasiaan data, *block cipher* juga menjadi bahan penyusun primitif lain, seperti *stream cipher* (melalui mode OFB dan CTR), *message authentication code* (misalnya CMAC), dan skema *authenticated encryption* [21]. Karena perannya yang mendasar tersebut, pemahaman tentang cara sebuah *block cipher* dirancang, bukan hanya cara memakainya, menjadi bekal penting dalam mempelajari kriptografi.

Landasan perancangan *block cipher* modern berasal dari Shannon [21], yang merumuskan dua prinsip: *confusion*, yaitu menyembunyikan hubungan antara kunci dan *ciphertext*, dan *diffusion*, yaitu menyebarkan pengaruh setiap bit *plaintext* ke seluruh *ciphertext*. Shannon juga mengusulkan *product cipher*, yakni pengulangan operasi substitusi dan transposisi yang masing-masing lemah sehingga gabungannya menjadi kuat. Hampir semua *block cipher* yang dikenal saat ini, termasuk DES dan AES, adalah *iterated cipher* yang mengikuti gagasan tersebut.

Sejarah *block cipher* juga menunjukkan bahwa rancangan yang baik pada masanya dapat menjadi tidak memadai. Kunci DES yang hanya 56 bit kini rentan terhadap *exhaustive key search* seiring kemajuan daya komputasi [21], dan strukturnya menjadi sasaran kriptanalisis diferensial serta kriptanalisis linier [21]. Ukuran blok 64 bit pun terbukti bermasalah: serangan Sweet32 memanfaatkan *birthday bound* 2^{32} blok (sekitar 32 GB data) untuk memulihkan *plaintext* pada mode CBC [14]. Pelajaran dari kasus-kasus ini, yaitu kunci harus cukup panjang, blok harus cukup lebar, dan komponen nonlinier harus tahan terhadap serangan statistik, menjadi pertimbangan langsung dalam rancangan pada laporan ini.

Tugas ini meminta perancangan sebuah *block cipher* baru yang memenuhi prinsip *confusion* dan *diffusion*, memakai substitusi berbasis *S-box*, transposisi, serta minimal satu operasi tambahan, dijalankan dalam beberapa putaran dengan kunci putaran yang diturunkan dari kunci utama, dan tidak menyalin algoritma yang sudah ada. Laporan ini menyajikan Cipher Fayadh, sebuah *block cipher* dengan ukuran blok 128 bit dan kunci 128 bit, beserta implementasinya pada lima mode operasi (ECB, CBC, CFB, OFB, CTR), skema *padding*, dan verifikasi integritas data. Perlu ditekankan bahwa Cipher Fayadh dibuat untuk tujuan pembelajaran. Algoritma ini belum melalui kriptanalisis oleh pihak lain, sehingga tidak dimaksudkan untuk menggantikan standar seperti AES pada penggunaan nyata.

1.2 Review Block Cipher Sejenis

Bagian ini meninjau secara singkat beberapa *block cipher* yang menjadi pembanding, terutama dari sisi struktur, komponen nonlinier, dan mekanisme difusinya.

DES. *Data Encryption Standard* [21] beroperasi pada blok 64 bit dengan kunci efektif 56 bit dan 16 putaran jaringan Feistel. Fungsi putarannya mengekspansi 32 bit menjadi 48 bit, meng-XOR-kannya dengan kunci putaran, melewatkannya ke delapan *S-box* $6 \rightarrow 4$ bit, lalu menerapkan permutasi P. Struktur Feistel membuat enkripsi dan dekripsi memakai rangkaian yang sama dengan urutan kunci dibalik [21]. Kelemahan utamanya adalah kunci dan blok yang terlalu pendek untuk ukuran saat ini [21, 14].

AES. *Advanced Encryption Standard* [21, 18], yang berasal dari algoritma Rijndael, memakai blok 128 bit dengan kunci 128, 192, atau 256 bit dan 10, 12, atau 14 putaran. AES bukan Feistel, melainkan *substitution-permutation network* (SPN): seluruh blok diproses pada setiap putaran melalui SubBytes (satu *S-box* 8×8 berbasis invers di $GF(2^8)$), ShiftRows, MixColumns, dan AddRoundKey. Rancangannya mengikuti *wide trail strategy*, yang memberi batas terbukti terhadap kriptanalisis diferensial dan linier. Konsekuensinya, setiap komponen harus memiliki invers dan dekripsi memerlukan rangkaian operasi tersendiri.

Blowfish. Blowfish [20] adalah Feistel 16 putaran dengan blok 64 bit dan kunci 32 sampai 448 bit. Fungsi putarannya memakai empat *S-box* $8 \rightarrow 32$ bit yang isinya bergantung pada kunci, lalu menggabungkan keluarannya dengan penjumlahan modulo 2^{32} dan XOR. Blowfish menunjukkan bahwa pencampuran operasi dari grup aljabar yang berbeda dapat dipakai sebagai sumber nonlinieritas tambahan, gagasan yang sebelumnya dipakai pada IDEA. Ukuran blok 64 bit membuatnya rentan terhadap Sweet32 [14].

Camellia dan TEA. Camellia menunjukkan bahwa struktur Feistel tetap layak untuk blok 128 bit: 18 putaran untuk kunci 128 bit, dengan fungsi putaran berupa lapisan *S-box* 8×8 diikuti lapisan difusi linier. Di sisi lain, TEA adalah Feistel 64 putaran tanpa *S-box* sama sekali, yang hanya mengandalkan penjumlahan modulo 2^{32}, pergeseran, dan XOR, serta memakai konstanta turunan rasio emas agar setiap putaran berbeda.

Tabel 1 merangkum perbandingan parameter *cipher* tersebut dengan Cipher Fayadh.

Tabel 1: Perbandingan Cipher Fayadh dengan *block cipher* sejenis

Parameter	DES	AES-128	Blowfish	Camellia-128	Cipher Fayadh
Ukuran blok	64 bit	128 bit	64 bit	128 bit	128 bit
Ukuran kunci	56 bit	128 bit	32–448 bit	128 bit	128 bit
Jumlah putaran	16	10	16	18	16
Struktur	Feistel	SPN	Feistel	Feistel	Feistel
S-box	8 buah, $6 \to 4$ bit, tetap	1 buah, 8×8, invers $GF(2^8)$	4 buah, $8 \to 32$ bit, bergantung kunci	4 buah, 8×8, tetap	1 buah, 8×8, tetap, hasil pencarian acak
Difusi	Ekspansi E, permutasi P	ShiftRows, MixColumns	Penjumlahan mod 2^{32}, XOR	Lapisan linier per *byte* (XOR)	*Carry* penjumlahan mod 2^{32}, rotasi, permutasi bit
Operasi tambahan	Ekspansi bit	Perkalian di $GF(2^8)$	Penjumlahan mod 2^{32}	Fungsi FL/FL^{-1}	Penjumlahan dan pengurangan mod 2^{32}, rotasi
Fungsi putaran	Tanpa *state*	Tanpa *state*	Tanpa *state*	Tanpa *state*	Ber-*state* (akumulator dan umpan balik)

1.3 Gagasan dan Pendekatan Utama

Rancangan Cipher Fayadh bertolak dari tiga keputusan utama.

Blok dan kunci 128 bit. Ukuran blok 64 bit memenuhi batas minimum spesifikasi, tetapi *birthday bound*-nya terlalu kecil untuk mode berantai seperti CBC [14]. Dengan blok 128 bit, batas tersebut naik menjadi 2^{64} blok. Kunci 128 bit dipilih agar sepanjang ukuran blok dan berada jauh di luar jangkauan *exhaustive key search*.

Jaringan Feistel sebagai kerangka. Blok dibelah menjadi dua bagian 64 bit, L dan R, lalu diproses sebanyak 16 putaran:

$$L_{i+1} = R_i, \qquad R_{i+1} = L_i \oplus F(R_i, K_i). \tag{1}$$

Pada struktur ini fungsi F hanya menghasilkan nilai yang di-XOR-kan ke separuh blok lainnya, sehingga dekripsi cukup menghitung ulang F dengan urutan kunci putaran dibalik [21, 17]. Fungsi F tidak perlu memiliki invers. Sifat inilah yang dimanfaatkan: F dapat diisi dengan proses yang menyimpan *state* dan sulit dibalik, sesuatu yang tidak mungkin dilakukan pada SPN seperti AES. Secara teoretis, Luby dan Rackoff menunjukk-

an bahwa jaringan Feistel menghasilkan permutasi pseudoacak selama fungsi putarannya pseudoacak, sehingga upaya perancangan dapat difokuskan pada kualitas F.

Fungsi putaran berbasis modulasi sigma-delta. Gagasan pembeda Cipher Fayadh adalah meminjam struktur modulator sigma-delta ($\Sigma\Delta$) dari bidang pengolahan sinyal. Modulator $\Sigma\Delta$ terdiri atas tiga bagian: integrator yang menjumlahkan selisih antara masukan dan umpan balik, *quantizer* yang memetakan isi integrator menjadi keluaran, dan jalur umpan balik yang mengembalikan keluaran tersebut ke masukan. Ketiganya dipetakan menjadi operasi kriptografi pada *word* 32 bit:

$$acc \leftarrow acc + (x - feedback) \bmod 2^{32}, \tag{2}$$

$$q \leftarrow S(acc \oplus k), \tag{3}$$

$$feedback \leftarrow q. \tag{4}$$

Integrator menjadi akumulator dengan penjumlahan modulo 2^{32}, *quantizer* menjadi *S-box* 8×8 yang diterapkan pada keempat *byte* akumulator setelah dicampur dengan kunci, dan keluaran q diumpanbalikkan ke *word* berikutnya. Dengan demikian satu langkah sudah memuat tiga jenis operasi yang secara aljabar tidak saling kompatibel, yaitu penjumlahan modulo 2^{32}, XOR, dan substitusi tabel. *S-box* menjadi sumber utama *confusion*, sedangkan *carry* pada penjumlahan dan rantai umpan balik membawa pengaruh satu *word* ke *word* lainnya.

Pemetaan langsung tersebut memiliki dua kelemahan yang ditangani dalam rancangan:

1. Pengaruh hanya mengalir ke depan, yaitu *word* kedua bergantung pada *word* pertama tetapi tidak sebaliknya. Untuk itu F menjalankan dua lintasan, maju lalu mundur, dengan kunci yang ditukar pada lintasan mundur sehingga setiap *word* terkena kedua bagian kunci putaran.

2. Bit terendah hasil penjumlahan modulo 2^{32} berperilaku linier (setara dengan XOR), dan *carry* hanya merambat ke arah bit yang lebih tinggi. Untuk itu disisipkan rotasi bit di antara kedua lintasan, dan F diakhiri dengan permutasi bit 64 bit yang menyebarkan kedelapan bit dari setiap *byte* ke delapan *byte* yang berbeda.

Di luar fungsi putaran, dua pendekatan lain melengkapi rancangan. Pertama, *S-box* tidak diambil dari *cipher* lain, melainkan dibangkitkan secara deterministik dari sejumlah kandidat permutasi acak dan dipilih berdasarkan nonlinieritas tertinggi serta *differential uniformity* terendah, dua ukuran ketahanan terhadap kriptanalisis linier dan diferensial. Kedua, *key schedule* menurunkan 16 kunci putaran 64 bit dari kunci utama dengan gabungan rotasi, penjumlahan konstanta turunan rasio emas yang berbeda di tiap putaran, dan *S-box* yang sama, sehingga hubungan antarkunci putaran tidak linier.

Pada tingkat aplikasi, Cipher Fayadh dijalankan pada lima mode operasi standar [21] dengan *padding* PKCS#7 untuk mode ECB dan CBC. Untuk menjamin integritas data

digunakan CMAC yang dibangun di atas Cipher Fayadh sendiri dalam skema *Encrypt-then-MAC*, dengan kunci enkripsi dan kunci MAC yang diturunkan terpisah dari kunci utama. Rincian rancangan dibahas pada Bab 3, sedangkan pengujian sifat *avalanche* dan *strict avalanche criterion* [17, 18, 19] disajikan pada Bab 5.

2 Dasar Teori

Bab ini membahas teori yang dipakai dalam merancang algoritma *block cipher* pada Bab 3. Bab ini juga berisi konsep dasar *block cipher*, struktur Feistel dan SPN, *S-box* dan permutasi, mode operasi, *padding* dan *integrity verification*, lalu penelitian terkait, disertai konsep dari bidang lain, yakni modulasi sigma-delta.

Masing-masing komponen akan digunakan sebagai landasan perancangan algoritma di Bab 3, dengan mode operasi akan diimplementasikan pada Bab 4.

2.1 Konsep *Block Cipher*

Block cipher adalah algoritma enkripsi simetris yang memproses data dalam blok berukuran tetap sebesar n bit. Untuk setiap kunci K, fungsi enkripsi E_K memetakan satu blok plainteks $P \in \{0,1\}^n$ ke satu blok cipherteks $C \in \{0,1\}^n$, dan E_K harus berupa permutasi atas $\{0,1\}^n$ agar setiap cipherteks hanya berasal dari tepat satu plainteks. Untuk dekripsi, cari fungsi invers $D_K = E_K^{-1}$ dengan kunci yang sama.

Rancangan modern umumnya berbentuk *iterated cipher*: satu fungsi putaran ρ diterapkan berulang sebanyak r kali, dan tiap putaran memakai kunci putaran (*round key*) K_i yang dibangkitkan dari kunci utama oleh algoritma *key schedule*:

$$C = E_K(P) = \rho_{K_r} \circ \rho_{K_{r-1}} \circ \cdots \circ \rho_{K_1}(P). \tag{5}$$

Shannon [8] merumuskan dua sifat yang harus dimiliki sebuah cipher.

1. *Confusion*, yakni menyembunyikan hubungan antara kunci dan cipherteks, biasanya lewat substitusi nonlinear.

2. *Diffusion*, yakni menyebarkan pengaruh satu bit plainteks atau kunci ke banyak bit cipherteks, biasanya lewat transposisi dan pencampuran. Ukuran diffusion yang lazim dipakai adalah *avalanche effect*: mengubah satu bit *input* seharusnya mengubah sekitar separuh bit *output*.

Ukuran blok juga membatasi keamanan. Berdasarkan *birthday paradox*, dua blok cipherteks yang sama mulai muncul setelah sekitar $2^{n/2}$ blok. Untuk $n = 64$, hanya dibutuhkan sekitar 2^{32} blok atau 32 GiB data sebelum akhirnya pola dapat diterka dengan mode CBC [14].

2.2 Jaringan Feistel / SPN

Dua kerangka umum untuk membangun fungsi putaran adalah jaringan Feistel dan *substitution-permutation network* (SPN).

Jaringan Feistel. Blok dibelah menjadi dua separuh L dan R. Pada putaran ke-i [9]:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus F(R_{i-1}, K_i). \tag{6}$$

Putaran ini selalu dapat dibalik tanpa memandang isi F:

$$R_{i-1} = L_i, \qquad L_{i-1} = R_i \oplus F(L_i, K_i). \tag{7}$$

Dekripsi cukup menjalankan *ciphertext* ke dalam fungsi F kembali dengan urutan kunci putaran yang dibalik. Karena kita tidak memerlukan invers dari F dalam dekripsi, F tidak perlu bijektif. Konsekuensinya, pada tiap putaran hanya separuh blok yang berubah. Jadi, dibutuhkan lebih banyak putaran agar seluruh bit saling memengaruhi. DES [1] adalah contoh jaringan Feistel.

SPN. Seluruh blok diubah pada tiap putaran melalui lapisan pencampuran kunci, lapisan substitusi (*S-box*), dan lapisan permutasi atau difusi linear. Karena seluruh blok diubah, *diffusion* tercapai dalam putaran yang lebih sedikit. Akan tetapi, diperlukan invers F dalam dekripsi. Artinya, wajib dipilih F yang bijektif bila menggunakan SPN. AES [2] adalah contoh SPN.

Tabel 2: Perbandingan jaringan Feistel dan SPN

Aspek	Feistel	SPN
Bagian blok yang berubah per putaran	Separuh	Seluruhnya
Syarat fungsi putaran	Bebas, tidak perlu dapat dibalik	Tiap lapisan harus dapat dibalik
Kode dekripsi	Sama dengan enkripsi, kunci dibalik	Berbeda, memakai invers enkripsi
Contoh	DES	AES

2.3 *S-Box* dan Permutasi

S-box. *S-box* adalah pemetaan nonlinear $S : \{0,1\}^m \to \{0,1\}^m$ yang diimplementasikan sebagai tabel. Untuk $m = 8$, tabelnya berisi 256 entri satu byte. Kualitas sebuah *S-box* diukur dengan beberapa kriteria:

- **Bijektif**, yaitu setiap *output* muncul tepat sekali.
- **Titik tetap**, artinya nilai x dengan $S(x) = x$ sebaiknya tidak ada.

- ***Nonlinearity***, yaitu seberapa jauh *S-box* dari fungsi linear. Semakin tinggi nilainya, semakin tahan terhadap kriptanalisis linear [10].
- ***Differential uniformity***, yaitu seberapa merata perubahan *input* diteruskan menjadi perubahan *output*. Semakin rendah nilainya, semakin tahan terhadap kriptanalisis diferensial [11, 12].

Sebagai pembanding, *S-box* AES memiliki *nonlinearity* 112 dan *differential uniformity* 4 [2].

Permutasi. Permutasi bit memindahkan bit ke posisi lain tanpa mengubah nilainya. Operasi ini linear dan berperan penting untuk *diffusion*. Permutasi menyebarkan bit *output* satu *S-box* ke *S-box* yang berbeda pada putaran berikutnya. Pendekatan ini dipakai pula oleh cipher ringan PRESENT [13].

2.4 Mode Operasi (ECB, CBC, CFB, OFB, CTR)

Block cipher hanya memproses satu blok, sehingga untuk data yang lebih panjang dibutuhkan mode operasi [3]. Misalkan P_j dan C_j adalah blok ke-j plainteks dan cipherteks, IV vektor inisialisasi, dan E_K fungsi enkripsi blok:

$$
\begin{aligned}
&\text{ECB:} && C_j = E_K(P_j) \\
&\text{CBC:} && C_j = E_K(P_j \oplus C_{j-1}),\ C_0 = IV \\
&\text{CFB:} && C_j = P_j \oplus E_K(C_{j-1}),\ C_0 = IV \\
&\text{OFB:} && O_j = E_K(O_{j-1}),\ O_0 = IV,\ C_j = P_j \oplus O_j \\
&\text{CTR:} && C_j = P_j \oplus E_K(ctr + j)
\end{aligned}
$$

Tabel 3: Ringkasan sifat mode operasi

Mode	Perlu *padding*	Butuh IV	Catatan
ECB	Ya	Tidak	Blok sama menghasilkan cipherteks sama
CBC	Ya	Ya	Enkripsi berantai, tidak dapat diparalelkan
CFB	Tidak	Ya	Dekripsi juga memakai E_K, bukan D_K
OFB	Tidak	Ya	*Keystream* tidak bergantung data, dekripsi memakai E_K
CTR	Tidak	Ya (nilai awal *counter*)	Dapat diparalelkan, akses acak, dekripsi memakai E_K

2.5 Modulasi Sigma-Delta ($\Sigma\Delta$)

Modulasi sigma-delta adalah teknik yang dipakai pada konverter digital-ke-analog (DAC) dan analog-ke-digital (ADC) untuk merepresentasikan sinyal beresolusi tinggi dengan *output* beresolusi rendah [7]. Modulator orde pertama terdiri atas tiga komponen:

- **Akumulator** yang menjumlahkan selisih antara *input* dan *output* sebelumnya,
- **Kuantisasi** $Q(\cdot)$ yang membuang resolusi dan menghasilkan *output*,
- ***Feedback*** yang mengembalikan *output* ke akumulator.

Untuk *input* $x[n]$, *output* $y[n]$, dan akumulator $v[n]$:

$$v[n] = v[n-1] + x[n] - y[n-1], \qquad y[n] = Q(v[n]). \tag{8}$$

Jika galat kuantisasi $e[n] = y[n] - v[n]$, *output* dapat ditulis $y[n] = x[n] + e[n] - e[n-1]$. Artinya galat kuantisasi dibawa ke sampel berikutnya (*noise shaping*) dan tidak hilang begitu saja.

Dua sifat modulator ini menjadi dasar gagasan perancangan *cipher*. Pertama, akumulator dan umpan balik dibawa dari sampel ke sampel, sehingga *output* ke-n bergantung pada seluruh *input* sebelumnya. Mekanisme ini mirip dengan mode CBC atau CFB, tempat satu blok bergantung pada blok sebelumnya melalui IV dan umpan balik. Kedua, kuantisasi bersifat *lossy*, dalam artian *input* tidak dapat dipulihkan secara persis dari *output*. Sifat ini menjadi inspirasi awal rancangan, dengan harapan hilangnya informasi mempersulit penelusuran *input* dari *output*. Pada struktur Feistel, fungsi F tidak perlu memiliki invers, sehingga sifat semacam ini tidak mengganggu proses dekripsi.

2.6 *Padding & Integrity Verification*

Padding. Mode ECB dan CBC mensyaratkan panjang data kelipatan ukuran blok. Skema PKCS#7 [4] menambahkan N byte bernilai N, dengan $1 \leq N \leq 16$ untuk blok 16 byte. Jika data sudah kelipatan blok, satu blok penuh tetap ditambahkan sehingga *padding* selalu dapat dibuang secara tidak ambigu.

Integritas. Enkripsi saja tidak menjamin data tidak diubah. Dibutuhkan *message authentication code* (MAC). CMAC [5] membangun MAC dari *block cipher*: dua subkunci K_1 dan K_2 diturunkan dari $E_K(0^{128})$ lewat *doubling* di $GF(2^{128})$, lalu data diproses secara berantai seperti CBC dan blok terakhir di-XOR dengan K_1 atau K_2. Pada skema *encrypt-then-MAC*, MAC dihitung atas cipherteks dan harus diverifikasi sebelum dekripsi. Kunci enkripsi dan kunci MAC sebaiknya dipisah (*key separation*).

2.7 *Related Works*

- **DES** [1]: Feistel 16 putaran, blok 64 bit, kunci 56 bit. Ukuran blok dan kunci yang kecil membuatnya tidak lagi memadai.

- **AES** [2]: SPN dengan blok 128 bit dan 10, 12, atau 14 putaran, memakai *S-box* aljabar, *ShiftRows*, dan *MixColumns*.
- **PRESENT** [13]: SPN ringan 64 bit dengan lapisan permutasi bit untuk menyebarkan *output S-box*.
- **Sigma-delta dalam enkripsi**: pada penelusuran kami, modulator sigma-delta ditemukan dalam konteks enkripsi bukan sebagai fungsi putaran *block cipher*, yaitu pada enkripsi data hasil modulator delta/sigma-delta dengan bit acak berbasis *chaos* [15] dan pada *quantum noise stream cipher* berbasis modulator sigma-delta dan *optical chaos* [16]. Kami tidak menemukan karya yang memakai prinsip sigma-delta sebagai fungsi putaran *block cipher*, namun penelusuran ini terbatas dan belum mencakup seluruh data jurnal yang ada.

3 Perancangan Block Cipher

Bab ini menjelaskan rancangan Cipher Fayadh: parameter, alur enkripsi dan dekripsi, pembentukan kunci putaran, *S-box*, permutasi dan rotasi, serta fungsi putaran berbasis sigma-delta.

3.1 Parameter Umum

Cipher Fayadh adalah *block cipher* Feistel 16 putaran dengan blok dan kunci 128 bit. Fungsi putarannya meminjam struktur modulator sigma-delta (Subbab 2.4).

Tabel 4: Parameter Cipher Fayadh

Parameter	Nilai
Ukuran blok	128 bit (16 byte)
Ukuran kunci utama	128 bit (16 byte)
Struktur	Jaringan Feistel dua cabang, masing-masing 64 bit
Jumlah putaran	16
Kunci putaran	16 kunci, masing-masing 64 bit
Ukuran *word* pada fungsi F	32 bit
S-box	8 bit, 256 entri, bijektif
Permutasi bit	64 bit, $i \mapsto 41\,i \bmod 64$

Ukuran blok 128 bit dipilih karena blok 64 bit rentan kolisi pada data besar (Subbab 2.1), dan ukuran kunci 128 bit memenuhi syarat spesifikasi bahwa kunci utama minimal sepanjang ukuran blok.

Tabel 5 memetakan syarat spesifikasi ke komponen rancangan.

Tabel 5: Pemenuhan syarat rancangan

Syarat	Komponen
Substitusi (*S-box*)	Tabel 256 entri pada fungsi F dan *key schedule*
Transposisi (permutasi)	Permutasi bit 64 bit di akhir fungsi F
Operasi tambahan	Penjumlahan modulo 2^{32} dan rotasi
Iterated cipher dengan kunci putaran	16 putaran, kunci putaran dari *key schedule*
Confusion	*S-box* dan pencampuran kunci
Diffusion	Akumulator sigma-delta, rotasi, permutasi bit, dan struktur Feistel

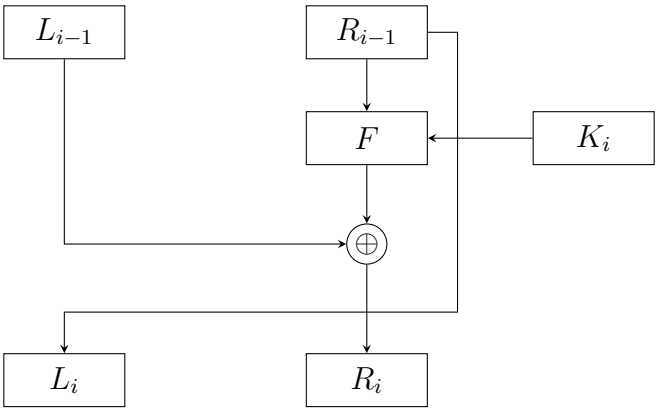
3.2 Alur Enkripsi

Blok plainteks 16 byte dibelah menjadi L_0 (8 byte kiri) dan R_0 (8 byte kanan), lalu untuk $i = 1, \ldots, 16$:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus F(R_{i-1}, K_i). \tag{9}$$

Setelah putaran ke-16, kedua separuh ditukar sekali lagi dan digabung menjadi cipherteks:

$$C = R_{16} \,\|\, L_{16}. \tag{10}$$



Gambar 1: Satu putaran Feistel pada Cipher Fayadh

3.3 Alur Dekripsi

Dekripsi memakai prosedur yang sama dengan enkripsi, dengan urutan kunci putaran dibalik $(K_{16}, K_{15}, \ldots, K_1)$. Pertukaran terakhir pada enkripsi membuat kedua prosedur identik. Jika cipherteks (R_{16}, L_{16}) dimasukkan ke putaran pertama dekripsi dengan K_{16}:

$$R_{16} \oplus F(L_{16}, K_{16}) = \big(L_{15} \oplus F(R_{15}, K_{16})\big) \oplus F(R_{15}, K_{16}) = L_{15}, \tag{11}$$

karena $L_{16} = R_{15}$. Keadaan setelah putaran ini adalah (R_{15}, L_{15}), yaitu keadaan putaran ke-15 enkripsi dalam keadaan tertukar. Pola yang sama berlanjut hingga setelah 16

putaran keadaannya (R_0, L_0), dan pertukaran terakhir mengembalikan (L_0, R_0), yaitu plainteks. Karena yang dikombinasikan hanya hasil F dan XOR, fungsi F tidak perlu memiliki invers.

3.4 Pembentukan *Key Schedule*

Kunci utama 128 bit dibelah menjadi dua *word* 64 bit a_0 dan b_0. Untuk $i = 1, \ldots, 16$:

$$\begin{aligned} a' &= S_{64}\big(\mathrm{rotl}_{64}(a_{i-1}, 13) + \varphi \cdot i \bmod 2^{64}\big), \\ b' &= b_{i-1} \oplus \mathrm{rotl}_{64}(a', 29), \\ (a_i, b_i) &= (b', a'), \\ K_i &= a_i \oplus b_i, \end{aligned}$$

dengan $\varphi =$ `0x9E3779B97F4A7C15` (turunan *golden ratio*), rotl_{64} rotasi kiri 64 bit, dan S_{64} substitusi tiap byte memakai *S-box*.

Perancangan ini bertujuan agar:

- tiap kunci putaran berbeda walau kunci utama berpola (mis. semua nol), melalui konstanta $\varphi \cdot i$ yang berubah tiap putaran;
- hubungan antar kunci putaran tidak linear terhadap kunci utama, melalui *S-box* dan penjumlahan modulo 2^{64};
- setiap K_i bergantung pada seluruh 128 bit kunci utama, karena a dan b saling mencampur lewat XOR dan pertukaran.

Nilai rotasi 13 dan 29 sengaja dipilih (bukan kelipatan 8), sehingga bit dalam satu byte terpecah ke byte yang berbeda.

3.5 *S-Box*

Tabel *S-box* 8 bit dibangkitkan secara deterministik oleh program `tools/gen_sbox.cpp`: tabel identitas diacak dengan *Fisher-Yates shuffle* menggunakan PRNG xorshift64 dengan *seed* 3141592653589793 (digit π), dan diulang 10.000 kali. Kandidat dengan titik tetap dibuang, kemudian dipilih kandidat dengan *nonlinearity* tertinggi dan, jika sama, *differential uniformity* terendah.

Tabel 6: Properti *S-box* Cipher Fayadh dibanding pembanding

Metrik	Cipher Fayadh	AES
Bijektif	ya	ya
Titik tetap $S(x) = x$	0	0
Nonlinearity	98	112
Differential uniformity	10	4

S-box ini bersifat bijektif dan tanpa titik tetap, tetapi kualitas nonlinear dan diferensialnya berada di kisaran *S-box* acak (*nonlinearity* 98 dan *differential uniformity* 10) dan di bawah *S-box* AES yang dikonstruksi secara aljabar. Ini keterbatasan pendekatan pencarian acak. Pada Cipher Fayadh, keamanan terhadap kriptanalisis linear dan diferensial tidak bertumpu pada *S-box* saja, tetapi pada kombinasinya dengan akumulator, rotasi, permutasi, dan 16 putaran.

3.6 Permutasi / Rotasi

Permutasi bit. Bit pada posisi i ($0 \le i < 64$) dipindah ke posisi

$$\pi(i) = 41\, i \bmod 64. \tag{12}$$

Karena $\gcd(41, 64) = 1$, π bijektif. Pengali dipilih agar 8 bit dari satu byte tersebar ke sebanyak mungkin byte tujuan, karena tugas utama permutasi di sini adalah memindahkan bit lintas byte (*S-box* bekerja per byte). Perbandingan beberapa pengali ganjil:

Tabel 7: Penyebaran 8 bit satu byte ke byte tujuan untuk beberapa pengali

Pengali	Byte tujuan berbeda (maks. 8)	Bit yang tidak berpindah
37	7	4
41	**8**	8
43	3	2
47	5	2
53	7	4
59	6	2

Pengali 41 adalah satu-satunya yang menyebarkan setiap byte ke 8 byte berbeda. Kelemahannya, 8 bit (posisi kelipatan 8) tidak berpindah pada lapisan ini. Hal itu tidak fatal karena bit tersebut tetap diolah *S-box*, penjumlahan, dan rotasi pada putaran berikutnya.

Rotasi. Rotasi kiri 32 bit dipakai di antara dua *pass* sigma-delta (Subbab 3.7), dengan jumlah 7 untuk *word* pertama dan 19 untuk *word* kedua. Rotasi diperlukan karena bit paling kanan hasil penjumlahan hanya bergantung pada bit paling kanan operan (setara XOR), sehingga mudah ditebak. Rotasi memindahkannya ke posisi yang ikut terkena *carry* penjumlahan pada langkah berikutnya.

3.7 Fungsi Putaran

Fungsi $F(R, K)$ menerima separuh blok R (64 bit) dan kunci putaran K (64 bit), dan menghasilkan masker 64 bit. R dan K dipecah menjadi dua *word* 32 bit, $R = (w_0, w_1)$ dan $K = (k_0, k_1)$ dengan indeks 0 sebagai bagian atas.

Langkah sigma-delta. Satu langkah memproses satu *word* x dengan kunci k, membawa akumulator a dan umpan balik q_{prev}:

$$\begin{aligned} a &\leftarrow a + (x - q_{\text{prev}}) \bmod 2^{32}, \\ q &= S_{32}(a \oplus k), \\ q_{\text{prev}} &\leftarrow q, \end{aligned}$$

dengan S_{32} substitusi tiap byte memakai *S-box*. *Output* langkah ini adalah q. Nilai awal a dan q_{prev} adalah 0 pada tiap *pass*.

Urutan fungsi F.

1. ***Pass* maju**: $w_0 \leftarrow \text{SD}(w_0, k_0)$, kemudian $w_1 \leftarrow \text{SD}(w_1, k_1)$ dengan akumulator dan umpan balik dibawa dari w_0.
2. **Rotasi**: $w_0 \leftarrow \text{rotl}_{32}(w_0, 7)$, $w_1 \leftarrow \text{rotl}_{32}(w_1, 19)$.
3. ***Pass* mundur**: $w_1 \leftarrow \text{SD}(w_1, k_0)$, kemudian $w_0 \leftarrow \text{SD}(w_0, k_1)$, kunci ditukar agar tiap *word* terkena kedua kunci, dan akumulator dimulai dari 0 lagi.
4. **Permutasi**: hasil $w_0 \,\|\, w_1$ dipermutasi dengan π pada Subbab 3.6.

Tabel 8: Pemetaan konsep sigma-delta ke fungsi F

Modulator sigma-delta	Fungsi F Cipher Fayadh
Akumulator	$a \leftarrow a + (x - q_{\text{prev}}) \bmod 2^{32}$
Kuantisasi	Diganti *S-box* dengan XOR kunci (bijektif, tidak *lossy*)
Umpan balik	$q_{\text{prev}} \leftarrow q$
Aliran satu arah	Dijalankan maju lalu mundur agar dua arah
(tidak ada)	Rotasi antar *word* dan permutasi bit

Hubungan dengan modulator sigma-delta. Rancangan ini hanya terinspirasi oleh modulator sigma-delta, bukan benar-benar *copy one-to-one*. Kuantisasi aslinya membuang resolusi, yang cocok untuk DAC, tetapi tidak untuk cipher (menganggu *integrity*). Sehingga, pada rancangan ini, diganti dengan *S-box* bijektif. Yang dipertahankan adalah sifat bahwa tiap *output* bergantung pada seluruh *input* sebelumnya melalui state yang dibawa, mirip mode CBC atau CFB tempat satu blok bergantung pada blok sebelumnya.

4 Implementasi dan Simulasi

Bab ini menguraikan implementasi Cipher Fayadh dalam aplikasi *Command Line Interface* (CLI), hasil pengujian simulasi enkripsi dan dekripsi menggunakan berkas uji yang *reproducible*, mekanisme penanganan *padding*, serta verifikasi integritas data berbasis *Encrypt-then-MAC*.

4.1 Deskripsi Program

Aplikasi CLI (`cipher_cli`) dibangun menggunakan C++17 tanpa *dependency* pustaka kriptografi pihak ketiga. Komponen inti (fungsi putaran F, ekspansi kunci, lima mode cipher, *padding*, dan CMAC) diimplementasikan langsung pada kode sumber, dengan pemanfaatan OpenMP untuk mempercepat mode ECB, CTR, serta dekripsi CBC dan CFB.

4.1.1 Antarmuka Perintah dan Format Kontainer

Program menyediakan tiga perintah operasional:

- `keygen [-f] -o <path>`: Membangkitkan kunci acak 128 bit (32 hex) menggunakan CSPRNG sistem operasi (`getentropy`) dengan izin berkas POSIX `0600` (`S_IRUSR | S_IWUSR`).
- `encrypt [-f] -m <mode> (-k <hex> | -key-file <path>) [-iv <hex>] (-i <in> | -t <teks>) -o <out>`: Mengenkripsi berkas atau teks masukan ke berkas keluaran.
- `decrypt [-f] (-k <hex> | -key-file <path>) -i <in> -o <out>`: Mendekripsi berkas kembali ke data aslinya (mode dibaca otomatis dari *header*).

Seluruh data terenkripsi dikemas dalam format biner **Container v1** dengan *overhead* tetap 38 byte.

Tabel 9: Format Kontainer v1

Field	Ukuran	Deskripsi
Magic Bytes	4 Byte	Pengenal format ASCII `"IF15"` (`0x49 0x46 0x31 0x35`)
Version	1 Byte	Nomor versi protokol (`0x01`)
Mode ID	1 Byte	ID mode: `0x01`=ECB, `0x02`=CBC, `0x03`=CFB, `0x04`=OFB, `0x05`=CTR
IV	16 Byte	Vektor inisialisasi / counter awal (diisi byte nol pada mode ECB)
Payload	N Byte	Data terenkripsi (N kelipatan 16 pada ECB/CBC, $N = L$ pada mode aliran)
CMAC Tag	16 Byte	Tag autentikasi integritas RFC 4493 atas *header* dan *payload*

Penyertaan *header* (Mode ID dan IV) ke dalam tag CMAC mencegah *Mode-Downgrade Attack* (penyerang memanipulasi byte mode ke ECB) dan *IV Bit-Flipping Attack* pada mode CBC.

4.1.2 Konvensi *Exit Code* dan Keamanan Runtime

Untuk memudahkan otomasi pengujian, `cipher_cli` menetapkan *exit code* terstruktur: `0` (sukses), `1` (kesalahan argumen/CLI), `2` (kesalahan I/O berkas), `3` (format rusak/`FormatError`), dan `4` (autentikasi gagal/`AuthError`).

4.2 Hasil Uji Enkripsi dan Dekripsi

Pengujian simulasi dilakukan menggunakan berkas uji yang telah tersedia di repositori (`test-results/samples/`) agar hasil bersifat deterministik dan *reproducible*:

- **Berkas Teks**: `teks.txt` (1.116 byte, SHA-256: `0118caf99b62bdf9...`), memuat teks multibaris, karakter non-ASCII, dan panjang bukan kelipatan 16.
- **Berkas Biner**: `gambar.png` (949 byte, SHA-256: `5acafa9454163dc6...`), berkas citra format PNG.

Pengujian menggunakan pasangan kunci dan IV tetap:

$$K_{\text{master}} = \texttt{000102030405060708090a0b0c0d0e0f}$$
$$\text{IV} = \texttt{0f0e0d0c0b0a09080706050403020100} \quad \text{(khusus non-ECB)}$$

Prosedur eksekusi enkripsi, dekripsi, dan verifikasi kesamaan biner pada terminal dapat direproduksi melalui perintah:

```
# Enkripsi dan dekripsi berkas teks pada mode CBC
./cipher_cli encrypt -m cbc -k 000102030405060708090a0b0c0d0e0f --iv 0f0e0d0c0b0a09080706050403020100 -i test-results/samples/teks.txt -o out.enc
./cipher_cli decrypt -k 000102030405060708090a0b0c0d0e0f -i out.enc -o out.dec

# Verifikasi kesamaan bit-demi-bit
cmp -s test-results/samples/teks.txt out.dec && echo "Identik 100%"
```

4.2.1 Berkas Teks

Pada berkas `teks.txt` (1.116 byte), mode ECB dan CBC menerapkan *padding* PKCS#7 sehingga *payload* menjadi 1.120 byte dan kontainer berukuran 1.158 byte ($22+1.120+16$). Pada mode CFB, OFB, dan CTR, *payload* tepat 1.116 byte dan kontainer berukuran 1.154 byte ($22+1.116+16$). Seluruh hasil dekripsi pada kelima mode terverifikasi 100% identik dengan berkas sumber via perintah `cmp`.

4.2.2 Berkas Biner

Pada berkas `gambar.png` (949 byte), mode ECB dan CBC menghasilkan *payload* 960 byte dan kontainer 998 byte. Mode aliran menghasilkan *payload* 949 byte dan kontainer 987

byte. Seluruh hasil dekripsi memiliki nilai hash SHA-256 yang identik dan gambar PNG dapat dibuka secara visual tanpa kerusakan struktur *chunk*.

Rangkuman kuantitatif hasil pengujian *round-trip* disajikan pada Tabel 10.

Tabel 10: Hasil Uji Round-Trip Enkripsi dan Dekripsi Berkas Uji

Berkas	Mode	Plainteks	Kontainer	Payload	Status Dekripsi
`teks.txt`	ECB	1.116 B	1.158 B	1.120 B	Identik (`pass`)
`teks.txt`	CBC	1.116 B	1.158 B	1.120 B	Identik (`pass`)
`teks.txt`	CFB	1.116 B	1.154 B	1.116 B	Identik (`pass`)
`teks.txt`	OFB	1.116 B	1.154 B	1.116 B	Identik (`pass`)
`teks.txt`	CTR	1.116 B	1.154 B	1.116 B	Identik (`pass`)
`gambar.png`	ECB	949 B	998 B	960 B	Identik (`pass`)
`gambar.png`	CBC	949 B	998 B	960 B	Identik (`pass`)
`gambar.png`	CFB	949 B	987 B	949 B	Identik (`pass`)
`gambar.png`	OFB	949 B	987 B	949 B	Identik (`pass`)
`gambar.png`	CTR	949 B	987 B	949 B	Identik (`pass`)

4.3 Penanganan Padding

Mode berbasis blok (ECB dan CBC) mewajibkan data merupakan kelipatan dari ukuran blok (16 byte). Program mengimplementasikan skema *padding* PKCS#7 sesuai RFC 5652 [4]. Untuk panjang data L byte, jumlah byte *padding* p bernilai:

$$p = 16 - (L \bmod 16) \tag{13}$$

di mana setiap byte *padding* bernilai heksadesimal p:

- `teks.txt`: $L = 1.116 \implies 1.116 \bmod 16 = 12 \implies p = 4$. Ditambahkan 4 byte `0x04 0x04 0x04 0x04`.
- `gambar.png`: $L = 949 \implies 949 \bmod 16 = 5 \implies p = 11$. Ditambahkan 11 byte `0x0B`.
- Data kelipatan 16 ($L \bmod 16 = 0$): Ditambahkan 1 blok penuh (16 byte `0x10`) untuk menjamin determinisme saat *unpadding*.

Sebaliknya, mode CFB, OFB, dan CTR beroperasi sebagai *stream cipher* yang melakukan XOR langsung antara *keystream* dan data, sehingga tidak memerlukan *padding* ($p = 0$).

4.4 Verifikasi Integritas Data

Integritas data dijamin menggunakan metode **Encrypt-then-MAC (EtM)** berbasis CMAC (RFC 4493) [5] dengan tag 128 bit (16 byte).

4.4.1 Pemisahan Kunci dan Perbandingan Waktu-Konstan

Sistem menerapkan dua prinsip perancangan kriptografi defensif:

1. **Pemisahan Kunci (*Key Separation*)**: Kunci enkripsi (K_{enc}) dan kunci MAC (K_{mac}) diturunkan secara terpisah dari K_{master} menggunakan fungsi pembangkit semu (PRF):

$$K_{\text{enc}} = \text{CIPH}_{K_{\text{master}}}(0^{15} \,\|\, 0\text{x}01), \quad K_{\text{mac}} = \text{CIPH}_{K_{\text{master}}}(0^{15} \,\|\, 0\text{x}02) \tag{14}$$

   Pemisahan ini menjamin independensi matematis antara kerahasiaan dan integritas data.

2. **Komparasi Waktu-Konstan**: Pengecekan tag CMAC menggunakan fungsi akumulasi bitwise OR branchless (`util::constant_time_equal`):

$$\Delta = \bigvee_{i=0}^{15} (T_{\text{dihitung}}[i] \oplus T_{\text{diterima}}[i]) \tag{15}$$

   Fungsi ini membutuhkan waktu eksekusi yang konstan tanpa keluar lebih awal (*early exit*), sehingga meniadakan serangan kanal samping waktu (*timing attack*).

4.4.2 Prinsip Fail-Stop

Pada proses dekripsi, tag CMAC diverifikasi **sebelum** mendekripsi *ciphertext* atau memeriksa *padding*. Jika verifikasi gagal, sistem langsung berhenti (*fail-stop*). Mekanisme ini mengeliminasi peluang terjadinya serangan *Padding Oracle* (Vaudenay) karena dekriptor tidak pernah memproses data yang terkorupsi.

5 Analisis Keamanan

Bab ini memaparkan hasil pengujian Cipher Fayadh sesuai spesifikasi: avalanche effect, entropi, dan histogram, masing-masing pada kelima mode operasi, ditambah beberapa pengujian lain. Seluruh data uji memakai *seed* tetap sehingga hasilnya dapat direproduksi dengan `make results`. Berkas log, CSV, dan grafik berada di folder `test-results/`.

5.1 Avalanche Effect

Avalanche effect mengukur persentase bit cipherteks yang berubah ketika satu bit plainteks atau satu bit kunci dibalik. Nilai ideal adalah 50%.

Satu blok. Pada tiap percobaan dipilih kunci dan plainteks acak, lalu 128 bit plainteks dan 128 bit kunci dibalik satu per satu dan dibandingkan dengan cipherteks semula. *Strict avalanche criterion* (SAC) melakukan pengujian dengan lebih ketat, di mana untuk

setiap pasangan bit *input* dan bit *output*, dihitung peluang bit *output* ikut berubah, yang seharusnya 50% pada seluruh 128×128 pasangan, bukan hanya rata-ratanya.

Tabel 11: Avalanche dan SAC pada satu blok

Uji	Percobaan	Rata-rata	Rentang
1 bit plainteks	1000×128	49,99%	27,34–69,53%
1 bit kunci	1000×128	49,99%	28,91–68,75%
SAC plainteks $\rightarrow$ cipherteks	2000	50,00%	45,75–54,30%
SAC kunci $\rightarrow$ cipherteks	2000	49,99%	45,70–54,20%

Rata-rata mendekati 50%. Rentang min–maks pada uji pertama dan kedua adalah sebaran per percobaan tunggal dan wajar untuk 128 bit. Pada SAC, tidak ada sel di luar 4σ dari 16.384 sel (data acak murni diperkirakan menghasilkan sekitar satu sel), sehingga tidak ada pasangan bit *input* dan *output* yang bias.

Per jumlah putaran. Pengujian yang sama diulang dengan membatasi cipher pada r putaran (500 percobaan per r).

Tabel 12: Avalanche rata-rata terhadap jumlah putaran

Putaran	1 bit plainteks	1 bit kunci
1	8,48%	21,15%
2	28,32%	46,14%
3	45,21%	49,97%
4	49,98%	49,99%
5	50,00%	50,00%
6	50,00%	49,97%
16	49,99%	50,00%

Difusi penuh tercapai pada putaran ke-4 untuk plainteks dan putaran ke-3 untuk kunci. Dengan 16 putaran, terdapat margin sekitar empat kali lipat dari titik itu. Lambatnya putaran pertama sesuai dengan sifat fungsi F: satu pemanggilan F hanya mengubah rata-rata sekitar 20 dari 64 bit *output* untuk perubahan 1 bit *input* (Subbab 5.4), karena *carry* penjumlahan hanya merambat ke bit yang lebih tinggi, dan pada struktur Feistel hanya separuh blok yang berubah per putaran.

Per mode operasi. Pengujian memakai pesan 64 blok (500 percobaan per mode). Satu bit plainteks atau satu bit kunci/IV dibalik, lalu diukur persentase bit cipherteks yang berubah pada blok yang sama, blok sebelumnya, dan blok sesudahnya.

Tabel 13: Avalanche per mode operasi

Mode	Blok sebelum	Blok yang sama	Blok sesudah	1 bit kunci	1 bit IV
ECB	0%	50,16%	0%	49,98%	tidak dipakai
CBC	0%	49,71%	50,01%	50,02%	49,96%
CFB	0%	0,78%	49,93%	49,99%	50,00%
OFB	0%	0,78%	0%	49,99%	49,97%
CTR	0%	0,78%	0%	49,95%	49,98%

Pada CFB, OFB, dan CTR, perubahan 1 bit plainteks hanya membalik 1 bit cipherteks di posisi yang sama (0,78% = 1/128). Ini merupakan sifat mode yang meng-XOR-kan plainteks dengan *keystream*. Perubahan 1 bit kunci atau IV tetap mengubah sekitar 50% seluruh cipherteks di semua mode. CBC dan CFB menyebarkan perubahan plainteks ke blok sesudahnya (sekitar 50%), sedangkan ECB membatasinya pada blok yang sama.

5.2 Entropi

Entropi Shannon byte cipherteks bernilai maksimum 8 bit/byte jika seluruh 256 nilai byte muncul sama sering. Selain entropi, uji χ^2 dengan 255 derajat kebebasan dipakai untuk menilai keseragaman: data seragam bernilai sekitar 255 ± 45 dan dianggap lolos jika $p \geq 0{,}05$. Tiap input berukuran 1 MiB dan hasilnya merupakan rata-rata dari 10 kunci dan IV.

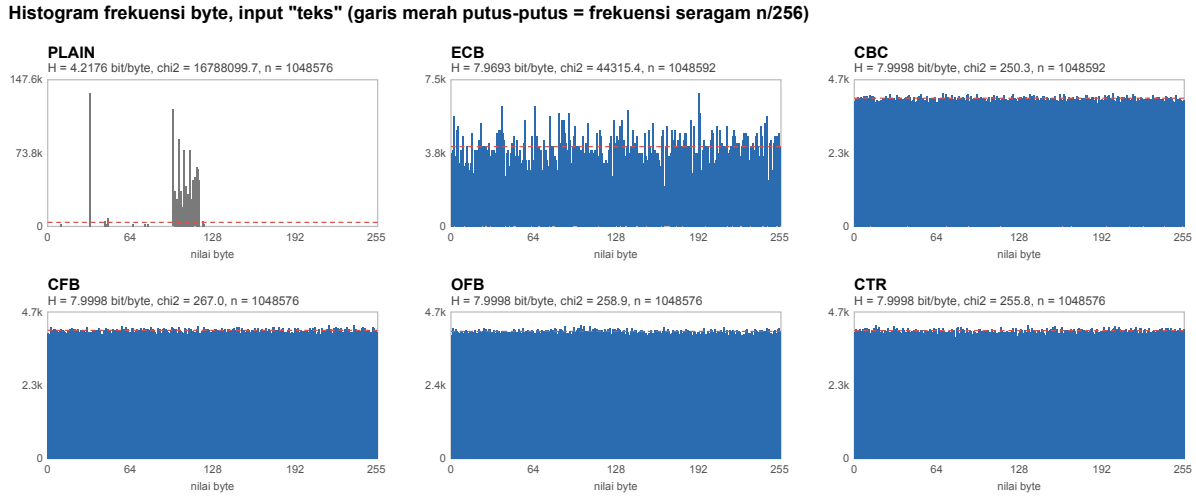
Tabel 14: Entropi (bit/byte) dan χ^2 pada input 1 MiB

Input	Ukuran	Plainteks	ECB	CBC	CFB	OFB	CTR
Teks berulang	Entropi	4,2176	7,9693	7,9998	7,9998	7,9998	7,9998
	χ^2	16.788.100	44.315	250,4	267,0	258,9	255,8
Semua nol	Entropi	0,0000	3,9205	7,9998	7,9998	7,9998	7,9998
	χ^2	267.386.880	17.196.378	239,0	255,9	250,5	259,6
Pola 0..255	Entropi	8,0000	7,1834	7,9998	7,9998	7,9998	7,9998
	χ^2	0,0	1.040.368	262,6	241,0	253,7	258,9

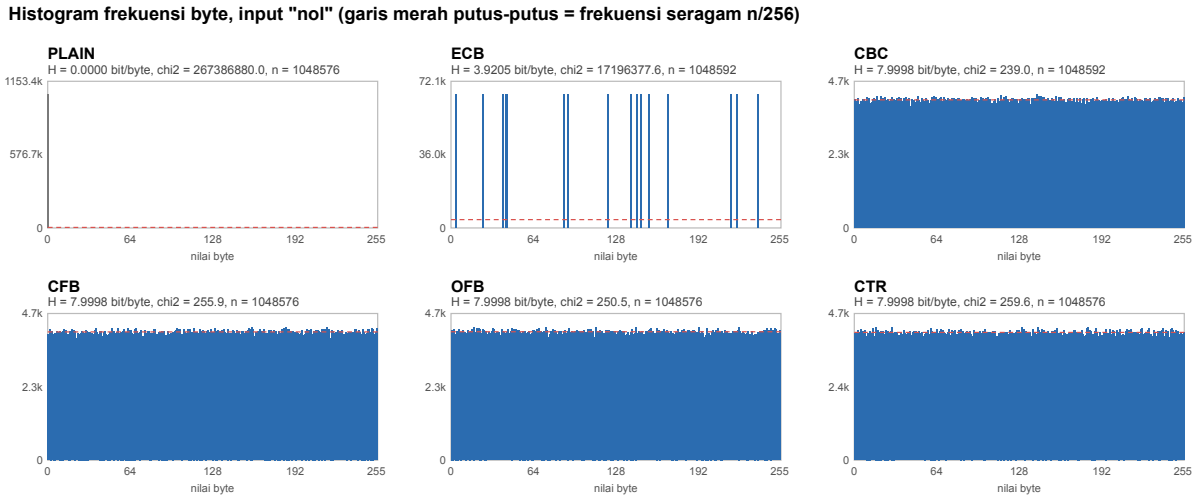
- CBC, CFB, OFB, dan CTR menghasilkan cipherteks yang secara statistik seragam pada ketiga input, dengan entropi 7,9998 bit/byte. Dari 120 percobaan (4 mode × 3 input × 10 kunci), 113 lolos uji χ^2 pada tingkat 5% (5,8% gagal), sesuai harapan untuk data acak yang memang gagal sekitar 5% secara kebetulan.
- ECB gagal pada semua input berpola karena blok plainteks yang sama menghasilkan blok cipherteks yang sama. Persentase blok 16 byte yang unik pada ECB tetap di bawah 0,6%, sedangkan mode lain 100%.
- Input "pola 0..255" menunjukkan bahwa entropi saja tidak cukup: plainteksnya memiliki entropi 8,0 (semua byte muncul sama sering) padahal sama sekali tidak acak. Uji χ^2 dan keunikan blok menangkap hal ini.

5.3 Histogram

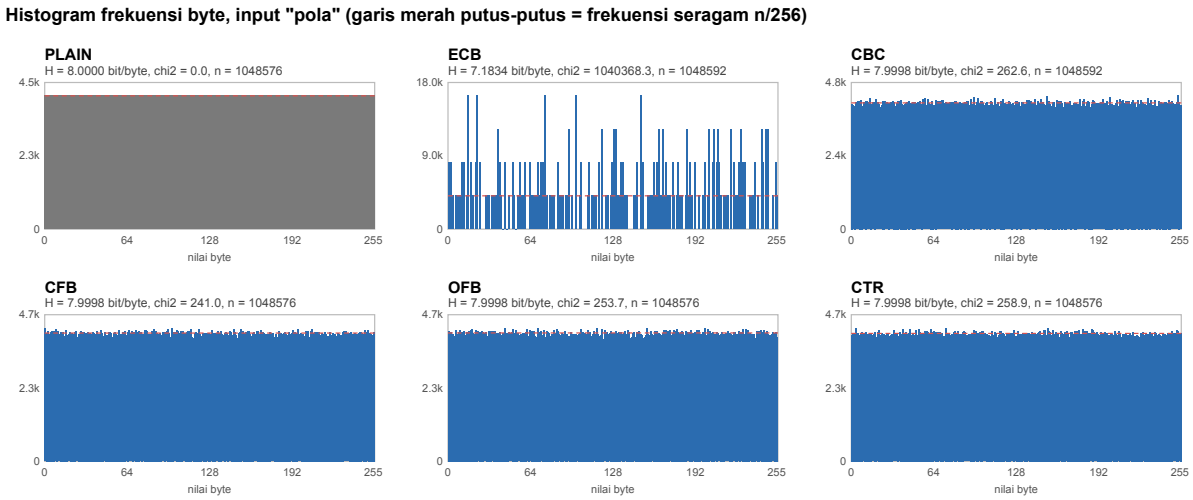
Gambar 2 sampai Gambar 4 membandingkan sebaran frekuensi byte plainteks dengan cipherteks pada kelima mode untuk tiga input. Garis putus-putus merah menandai frekuensi seragam ($n/256$).



Gambar 2: Histogram byte, input teks berulang



Gambar 3: Histogram byte, input semua nol



Gambar 4: Histogram byte, input pola 0..255

Plainteks terkonsentrasi pada beberapa nilai byte, sedangkan cipherteks CBC, CFB, OFB, dan CTR mendekati distribusi *uniform* pada ketiga input. Histogram ECB masih menyimpang dari seragam (pada input teks terlihat gundukan, pada input semua nol hanya sedikit nilai byte yang muncul) karena struktur plainteks masih terbawa ke cipherteks.

5.4 Pengujian Tambahan

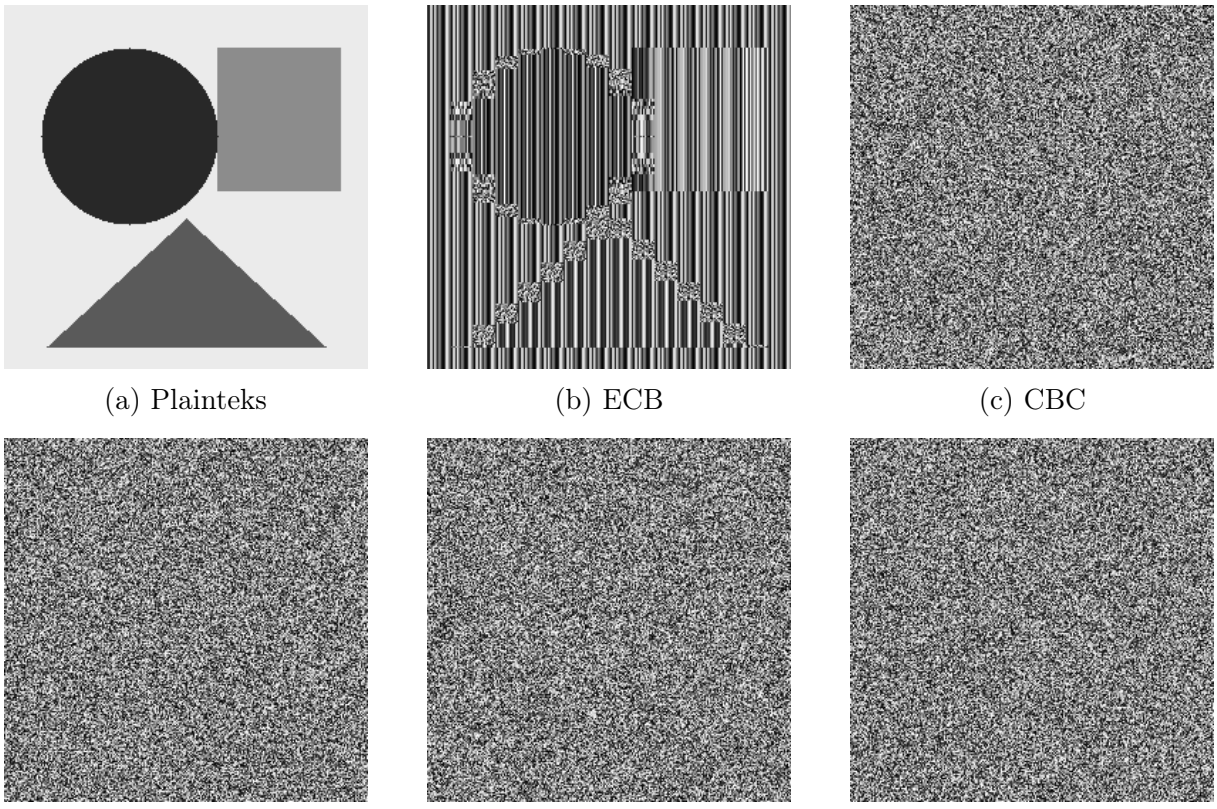
Properti fungsi F. Pengujian `tests/test_round_f.cpp` dilakukan untuk menelaah sifat fungsi F pada implementasi ini.

Tabel 15: Properti fungsi F (kunci tertentu)

Uji	Hasil
F *invertible* ($F^{-1}(F(R,K)) = R$)	100.000 dari 100.000 pasangan (R, K)
Tabrakan *output*, 100.000 *input* acak	0
Bit *output* berubah untuk 1 bit *input* (rata-rata)	19,77 dari 64
per posisi bit *input* (min–maks)	15,03–24,51 dari 64

Hasil ini menunjukkan bahwa F pada implementasi ini tidak kehilangan informasi, meskipun terinspirasi dari modulator sigma-delta yang bersifat *lossy* (kuantisasi diganti *S-box* bijektif). Difusi satu pemanggilan F sekitar 31%, jauh di bawah 50%, dan difusi penuh baru dicapai lewat putaran berulang seperti pada Tabel avalanche per putaran.

Demo mode pada gambar. Gambar grayscale 256×256 (1 byte per piksel, sehingga 1 blok = 16 piksel) dienkripsi pada tiap mode, lalu cipherteksnya ditampilkan kembali sebagai gambar.



(a) Plainteks (b) ECB (c) CBC

(d) CFB (e) OFB (f) CTR

Gambar 5: Gambar yang sama dienkripsi pada kelima mode

Bentuk gambar masih terlihat pada ECB, sedangkan keempat mode lain tampak seperti derau.

Verifikasi integritas. Pada pengujian lewat CLI, cipherteks yang dibalik 1 bit, kunci yang salah, dan kontainer yang terpotong semuanya ditolak tanpa menghasilkan berkas *output* (kode keluar 4 untuk dua kasus pertama dan 3 untuk kontainer terpotong). Pengujian ini dilakukan pada tiga lapis:

- skenario CLI pada `tests/run_all.sh`; hasilnya ada pada berkas `cli/roundtrip.md` di `test-results`;
- uji *end-to-end* `tests/e2e.sh`, yang membalik 1 bit pada *magic*, IV, cipherteks, dan *tag* untuk tiap mode, serta menguji kunci salah dan kontainer terpotong (60 pemeriksaan, log di `test-results/logs/e2e.log`);
- uji unit `tests/test_container.cpp`, yang membalik setiap bit pada kontainer satu per satu dan memastikan semuanya ditolak.

Kecepatan serial dan paralel. Mode yang dapat diparalelkan (ECB, dekripsi CBC dan CFB, serta CTR) diuji dengan data 64 MiB pada 16 *thread*. *Output* paralel identik dengan serial pada seluruh pengujian.

Tabel 16: Kecepatan serial dan paralel, data 64 MiB (median 5 kali)

Mode	Arah	Serial (MiB/s)	Paralel (MiB/s)	Speedup
ECB	enkripsi	16,44	128,96	7,84×
ECB	dekripsi	16,14	128,85	7,98×
CBC	enkripsi	16,09	15,94	0,99×
CBC	dekripsi	15,28	130,30	8,53×
CFB	enkripsi	16,54	16,24	0,98×
CFB	dekripsi	16,80	131,25	7,81×
OFB	enkripsi	14,97	15,85	1,06×
CTR	enkripsi	16,34	127,83	7,82×

Enkripsi CBC, CFB, dan OFB tidak dapat diparalelkan karena setiap blok bergantung pada blok sebelumnya, sehingga *speedup*-nya sekitar 1× (selisih kecil dari 1 berasal dari ukuran pengukuran). Pengukuran dilakukan pada Intel Core i5-12500H (Arch Linux, g++ 16.2.1, `-O2`).

Kualitas *S-box*. *S-box* bijektif, tanpa titik tetap, dengan *nonlinearity* 98 dan *differential uniformity* 10 (rincian dan pembanding AES pada Bab 3). Nilai ini dihitung oleh `tools/analyze_sbox.cpp` (`make analyze`, log di `test-results/logs/sbox.log`), yang tertulis $\max |W_S| = 60$, probabilitas diferensial maksimum 10/256, dan bias linear maksimum 60/512. Nilai ini berada dalam batas wajar untuk *S-box* acak terpilih dan lebih lemah dari *S-box* AES.

6 Kesimpulan dan Saran Pengembangan

6.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian pada laporan ini, diperoleh kesimpulan berikut.

1. **Rancangan memenuhi syarat spesifikasi.** Cipher Fayadh adalah *block cipher* jaringan Feistel 16 putaran dengan blok dan kunci 128 bit. Substitusi dilakukan oleh *S-box* 8 bit, transposisi oleh permutasi bit 64 bit, dan operasi tambahan berupa penjumlahan modulo 2^{32} serta rotasi. Setiap putaran memakai kunci putaran yang dibangkitkan dari kunci utama oleh *key schedule*.

2. **Implementasi berfungsi pada kelima mode operasi.** ECB, CBC, CFB, OFB, dan CTR dilengkapi *padding* PKCS#7 dan verifikasi integritas *encrypt-then-MAC* dengan CMAC. Pada seluruh pengujian, hasil dekripsi identik dengan berkas asli, sedangkan cipherteks yang diubah, kunci yang salah, dan kontainer yang terpotong ditolak.

3. **Difusi cipher memenuhi target.** Avalanche effect rata-rata 49,99% untuk perubahan 1 bit plainteks maupun kunci, dan uji SAC tidak menemukan pasangan bit *input* dan *output* yang bias (0 dari 16.384 sel di luar 4σ). Difusi penuh dicapai pada putaran ke-4, sehingga 16 putaran memberi margin sekitar empat kali lipat.

4. **Keacakan cipherteks bergantung pada mode.** CBC, CFB, OFB, dan CTR menghasilkan cipherteks yang mendekati seragam (entropi 7,9998 bit/byte) pada ketiga *input* uji, sedangkan ECB tetap membocorkan pola plainteks karena blok yang sama menghasilkan cipherteks yang sama. Hal ini sifat mode ECB dan bukan kelemahan cipher.

5. **Ada keterbatasan yang perlu dicatat.**

   - *S-box* bijektif dan tanpa titik tetap, tetapi *nonlinearity* 98 dan *differential uniformity* 10 lebih lemah daripada *S-box* AES (112 dan 4).
   - Belum dilakukan analisis ketahanan terhadap kriptanalisis diferensial dan linear (misalnya penghitungan lintasan), dan Cipher Fayadh belum dikriptanalisis oleh pihak lain. Hasil uji statistik pada Bab 5 tidak menggantikan analisis tersebut.
   - Konstanta rotasi dan pengali permutasi dipilih sebagai titik awal dan belum dicari secara sistematis.

6. **Inspirasi sigma-delta tercapai sebagian.** Rancangan fungsi F terinspirasi dari modulator sigma-delta dengan dua gagasan: (a) *state* yang dibawa dari satu *word* ke *word* berikutnya, dan (b) sifat *lossy* pada kuantisasi yang diharapkan mempersulit penelusuran *input* dari *output*. Gagasan (a) tercapai, karena *output* tiap langkah bergantung pada seluruh *input* sebelumnya dalam satu *pass*. Gagasan (b) belum tercapai: kuantisasi diganti *S-box* bijektif agar dapat menyertakan kunci, sehingga F pada implementasi ini masih dapat dibalik untuk kunci tertentu (100.000 dari 100.000 pasangan uji). Sifat *lossy* karenanya tidak dapat dijadikan alasan keamanan, dan ketahanan yang teramati berasal dari kunci, *S-box*, rotasi, permutasi, dan struktur Feistel. Gagasan ini dapat ditelaah kembali pada pengembangan selanjutnya.

6.2 Saran Pengembangan (Future Works)

- **Menelaah ulang kuantisasi *lossy*.** Merancang langkah kuantisasi yang benar-benar membuang sebagian informasi (misalnya memotong sebagian bit *output*) tanpa melemahkan difusi, lalu mengujinya kembali dengan avalanche dan SAC. Uji properti fungsi F pada Bab 5 perlu disesuaikan karena kondisinya akan berbalik.

- **Analisis kriptanalisis diferensial dan linear.** Menghitung jumlah *S-box* aktif dan batas probabilitas lintasan diferensial dan linear pada r putaran, sebagai pelengkap uji statistik.

- **Optimasi konstanta.** Mencari konstanta rotasi (7, 19, 13, 29) dan pengali permutasi secara sistematis berdasarkan avalanche per putaran, bukan sekadar titik awal.
- **Memperbaiki kualitas *S-box*.** Memakai konstruksi aljabar atau pencarian yang lebih ekstensif agar *nonlinearity* dan *differential uniformity* mendekati *S-box* AES.
- **Pengujian lebih luas.** Menambah uji keacakan statistik standar dan data uji yang lebih beragam, serta kriptanalisis oleh pihak lain sebelum Cipher Fayadh dapat dianggap layak dipakai di luar keperluan pembelajaran.

Pustaka

[1] National Institute of Standards and Technology, *FIPS PUB 46-3: Data Encryption Standard (DES)*, 1999. https://csrc.nist.gov/pubs/fips/46-3/final

[2] National Institute of Standards and Technology, *FIPS PUB 197: Advanced Encryption Standard (AES)*, 2001. https://csrc.nist.gov/pubs/fips/197/final

[3] M. Dworkin, *NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, National Institute of Standards and Technology, 2001. https://csrc.nist.gov/pubs/sp/800/38/a/final

[4] R. Housley, *RFC 5652: Cryptographic Message Syntax (CMS)*, IETF, 2009. https://www.rfc-editor.org/info/rfc5652

[5] J. Song, R. Poovendran, J. Lee, dan T. Iwata, *RFC 4493: The AES-CMAC Algorithm*, IETF, 2006. https://www.rfc-editor.org/info/rfc4493

[6] D. R. Stinson, *Cryptography: Theory and Practice*, edisi ke-3, Chapman & Hall/CRC, 2005. https://doi.org/10.1201/9781420057133

[7] R. Schreier, G. Temes, dan S. Pavan, *Understanding Delta-Sigma Data Converters*, edisi ke-2, Wiley-IEEE Press, 2017. https://doi.org/10.1002/9781119258308

[8] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949. https://doi.org/10.1002/j.1538-7305.1949.tb00928.x

[9] H. Feistel, "Cryptography and Computer Privacy," *Scientific American*, vol. 228, no. 5, pp. 15–23, 1973. https://doi.org/10.1038/scientificamerican0573-15

[10] M. Matsui, "Linear Cryptanalysis Method for DES Cipher," *EUROCRYPT '93*, LNCS 765, pp. 386–397, 1994. https://doi.org/10.1007/3-540-48285-7_33

[11] E. Biham dan A. Shamir, "Differential Cryptanalysis of DES-like Cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1991. https://doi.org/10.1007/BF00630563

[12] K. Nyberg, "Differentially Uniform Mappings for Cryptography," *EUROCRYPT '93*, LNCS 765, pp. 55–64, 1994. https://doi.org/10.1007/3-540-48285-7_6

[13] A. Bogdanov dkk., "PRESENT: An Ultra-Lightweight Block Cipher," *CHES 2007*, LNCS 4727, pp. 450–466, 2007. https://doi.org/10.1007/978-3-540-74735-2_31

[14] K. Bhargavan dan G. Leurent, "On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN," *ACM CCS*, 2016. https://doi.org/10.1145/2976749.2978423 (ringkasan: https://sweet32.info/)

[15] G. Ablay, "Chaotic Encryption Based Data Transmission Using Delta and Delta-Sigma Modulators," *International Journal of Applied Mathematics, Electronics and Computers*, 2016. https://dergipark.org.tr/en/pub/ijamec/article/280466

[16] H. Luo dkk., "Device-compatible ultra-high-order quantum noise stream cipher based on delta-sigma modulator and optical chaos," *Communications Engineering*, vol. 3, 2024. https://doi.org/10.1038/s44172-024-00171-x

[17] I K. Y. Sucipta, E. R. P. Lumban Tobing, dan S. A. Al Fata, "BASE: Block Cipher Feistel Berbasis Enhanced Logistic Map," *Jurnal Matematika Integratif*, vol. 21, no. 1, pp. 41–54, 2025. https://journals.unpad.ac.id/jmi/article/download/62475/pdf

[18] N. Angraini dan Y. Suryanto, "Modification Advanced Encryption Standard (AES) Algorithm with Perfect Strict Avalanche Criterion S-Box," *Jurnal Teknik Informatika (Jutif)*, vol. 3, no. 4, 2022. https://jutif.if.unsoed.ac.id/index.php/jurnal/article/view/352

[19] W. Prabowo dan N. Anwar, "Pengujian Model Simulasi Efek Avalanche Kriptografi Simetris Algoritma AES 128-bit, Mode ECB dan CBC," *IKRA-ITH Informatika: Jurnal Komputer dan Informatika*, 2025. https://journals.upi-yai.ac.id/index.php/ikraith-informatika/article/view/4775

[20] R. F. F. Siregar dkk., "Comparative Analysis of Encryption and Decryption Speed of AES and Blowfish Algorithms on Various File Sizes," *IJGAM*, vol. 2, no. 1, pp. 449–458, Januari 2026. https://jurnal-ijgam.or.id/index.php/IJGAM/article/download/119/146/463

[21] R. Munir, *IF4020 Kriptografi, Semester I Tahun 2026/2027*, Program Studi Teknik Informatika, STEI ITB. Diakses 13 Oktober 2026. https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/kripto26-27.htm

A Lampiran

A.1 Repositori Source Code

https://github.com/reletz/IF4020-15-Tugas2/

A.2 Video Demo Program

https://youtu.be/PXU4Q3QNZqc

A.3 Dokumentasi API

https://fayadh-crypto-api.naufarrel.dev

A.4 Pembagian Tugas

Nama	NIM	Kontribusi
Dita Maheswari	13523125	*S-box* dan analisis: `gen_sbox`, `analyze_sbox`, `sbox.cpp`.
Naufarrel Zhafif Abhista	13523149	Inti cipher: Feistel, *key schedule*, permutasi bit, fungsi F (*loop* $\Sigma\Delta$).
I Made Wiweka Putera	13523156	Integritas dan aplikasi: CMAC (`mac`), CLI (`main`), `util` (IV acak, baca/tulis berkas).
Hasri Fayadh Muqaffa	13523160	Mode operasi: `padding`, lima mode, OpenMP untuk CTR dan ECB; serta menyusun pengujian-pengujian proyek.